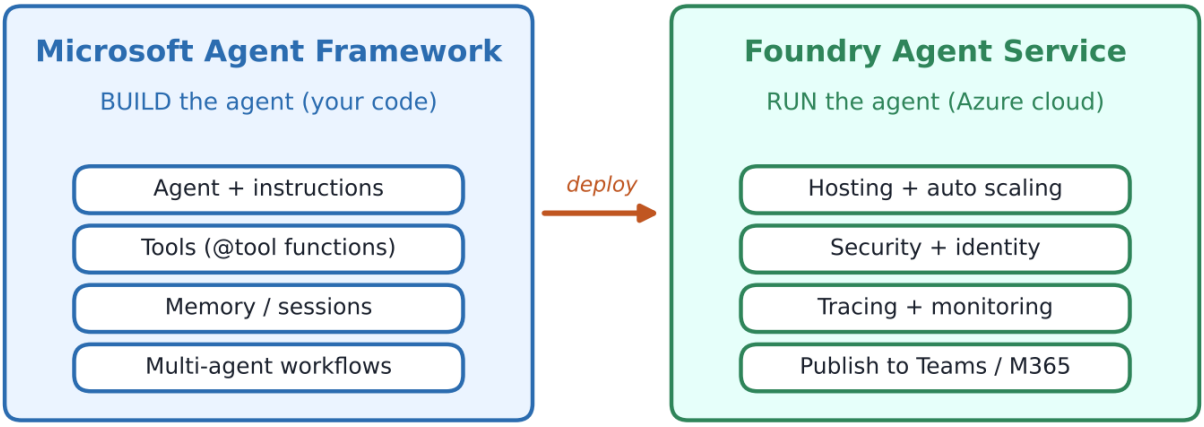
Microsoft Agent Framework & Foundry Agent Service

A beginner-friendly guide: what they are, how they work step by step, and how they fit together



The one-line summary: MAF is how you build an agent. Foundry Agent Service is where it can run in production.

Written November 2026 | Explained using a shopping-assistant example

What's inside

1. First, what is an AI agent?
2. Part A: Microsoft Agent Framework (MAF)
   What it is, where it came from, architecture, how a run works step by step, tools, multi-agent orchestration, and a complete code example
3. Part B: Microsoft Foundry Agent Service
   What it is, the three ways to build agents, architecture, lifecycle, security
4. How MAF and Foundry work together
5. Side-by-side comparison and when to use which
6. Interview questions and answers
7. Quick glossary and further reading

How to read this guide
Read it top to bottom once. The diagrams carry most of the ideas, so spend a minute on each one. Throughout the guide we use one running example: a shopping assistant that can search products and calculate order totals, using two tools called *search_products* and *calculate_order_total*.

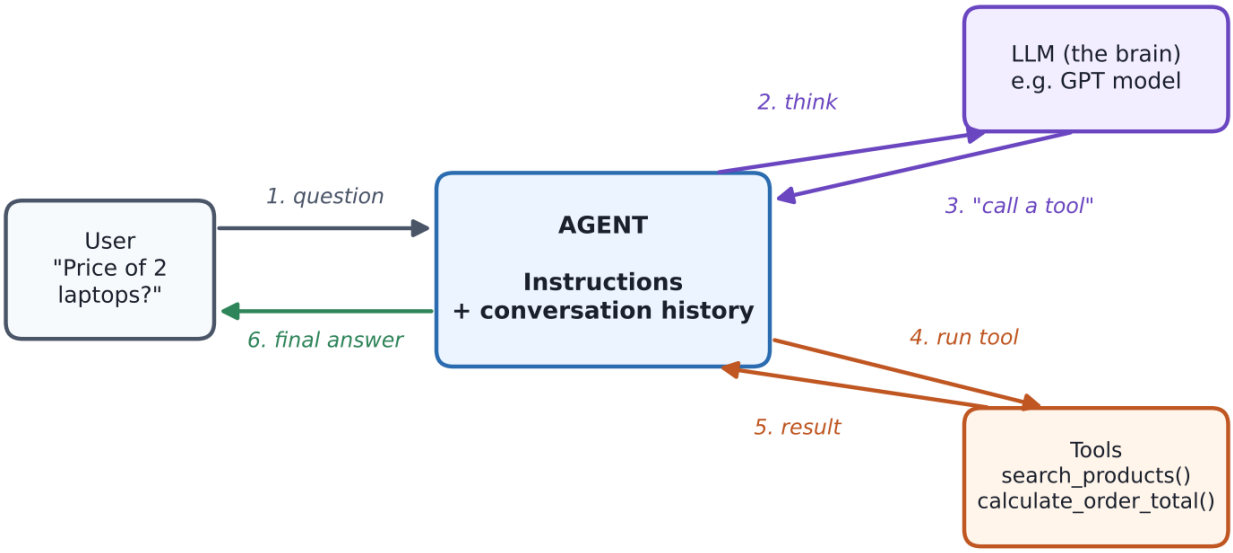
A quick analogy if you know FastAPI
FastAPI is a library you use to **write** a web API. A cloud service like Azure App Service is a place that **runs** it for you. MAF and Foundry Agent Service have the same relationship: MAF is the library for writing agents, and Foundry Agent Service is Azure's managed place to run them.

1. First, what is an AI agent?

A normal chatbot only talks. You ask a question, the language model writes an answer from what it already knows, and that's it. An **AI agent** can also **do things**. It can look up data, call an API, run a calculation, or hand work to another agent, and then use the results to answer you.

Every agent is made of three basic parts:

- **A model (LLM)**, the brain. For example, a GPT model. It reads the conversation and decides what to do next.
- **Instructions**, the job description. For example: "You are a shop assistant. Always check stock before quoting a price."
- **Tools**, the hands. Normal Python functions the model is allowed to call, like our *search_products*.



The agent loop. The model never runs your code itself; it asks for a tool, the framework runs it, and the result goes back to the model.

Notice steps 3 to 5. The model only *asks* for a tool call, something like "please call calculate_order_total with P101 and 2". The framework actually runs your Python function and sends the result back. The model may loop through this several times before it gives the final answer.

Writing all of this plumbing yourself (sending messages, reading tool requests, running functions, keeping history) is a lot of work. That's exactly the job of an agent framework.

2. Part A: Microsoft Agent Framework (MAF)

2.1 What it is

Microsoft Agent Framework is a free, open-source SDK from Microsoft for building AI agents and multi-agent systems in **Python** and **.NET (C#)**. You install it with pip, write normal code, and run it wherever you like: your laptop, your own server, or the cloud.

In simple words: **MAF is the toolkit you write your agent with.** It is code-first, which means you control everything through your own Python files rather than a web portal.

2.2 Where it came from

Before MAF, Microsoft had two separate projects. **Semantic Kernel** was the stable, enterprise-friendly SDK for adding AI to apps. **AutoGen** was a research project famous for getting several agents to talk and work together. Developers had to choose between stability and powerful multi-agent features.

MAF merges the two into one framework. The 1.0 version was released on April 3, 2026, with stable APIs and long-term support. AutoGen is now in maintenance mode, so new projects should start with MAF. The framework has kept moving quickly since then, with frequent minor releases adding new features.

2.3 The building blocks

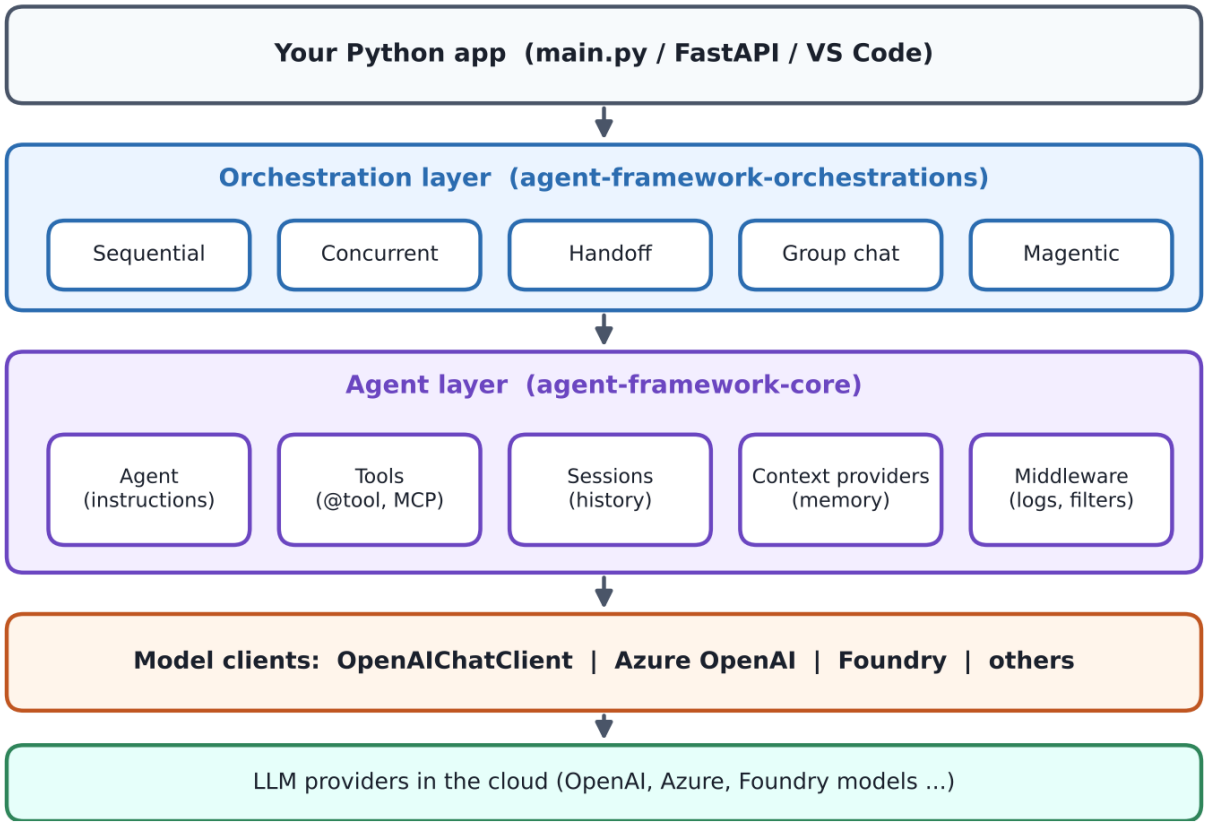
These are the main pieces you'll meet when you use MAF. Don't try to memorise them all at once; the architecture diagram on the next page shows how they stack up.

Building block	What it does	In our shopping example
Model client	Connects your agent to a language model and handles sending and receiving messages.	*OpenAIChatClient()* talks to a GPT model.
Agent	The agent itself: a name, instructions, a model client and a list of tools.	"ShopAssistant" with shop instructions.
Tools	Python functions the model can call. Marked with *@tool*. MAF can also use tools from MCP servers.	*search_products*, *calculate_order_total*
Sessions	Keep the conversation history so the agent remembers earlier messages in the same chat.	"Show me laptops" then "what about the second one?"
Context providers	Add memory or extra knowledge (for example, facts about the user) before each run.	Remembering the customer's city for delivery.

Building block	What it does	In our shopping example
Middleware	Code that runs around every agent call or tool call: logging, filtering, blocking unsafe input.	Logging every order calculation.
Workflows / orchestrations	Connect several agents or steps together in a graph.	A search agent plus a billing agent.

2.4 MAF architecture

Think of MAF as layers. Your app sits on top. Under it, the orchestration layer coordinates many agents when you need more than one. The agent layer holds the core pieces from the table. At the bottom, model clients translate everything into API calls to whichever LLM provider you choose.



MAF architecture. Each layer only talks to the one below it, so you can swap the model provider without rewriting your agent.

This layering is why you can switch from OpenAI to Azure OpenAI, or another provider, by changing one line: the model client. Your instructions, tools and workflows stay the same.

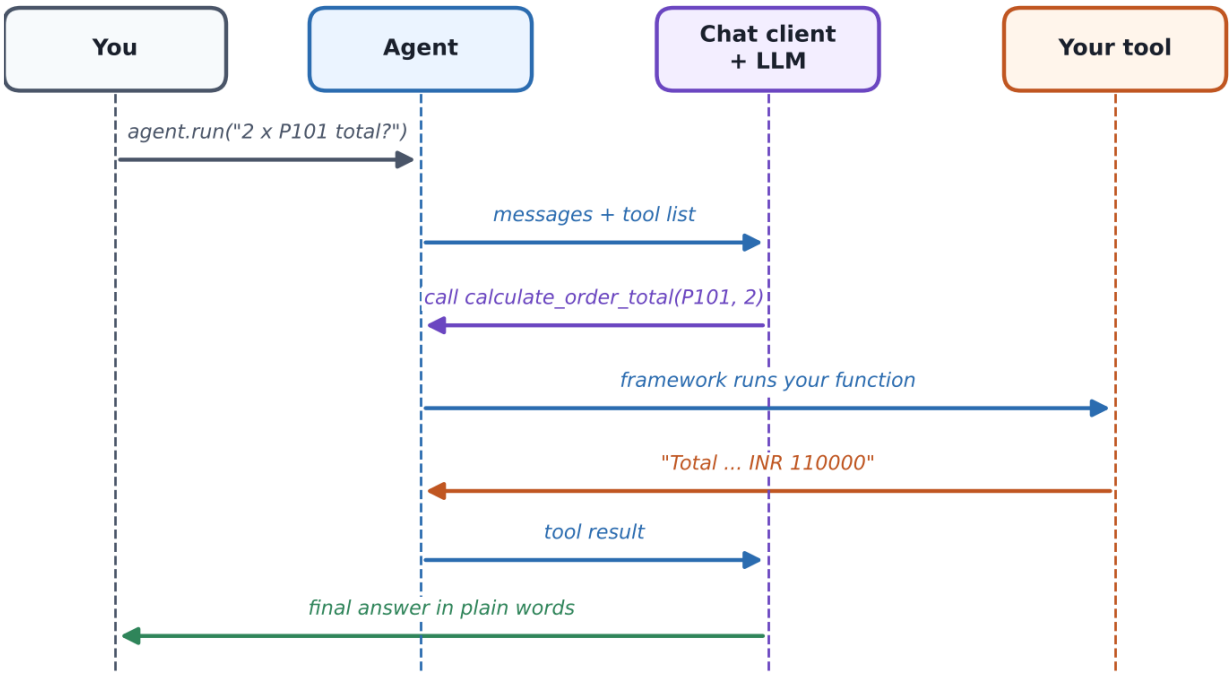
It also explains the package names you saw in a *requirements.txt*:

- **agent-framework-core**: the agent layer (Agent, @tool, sessions, middleware).
- **agent-framework-openai**: the OpenAI model client.

- **agent-framework-orchestrations**: the multi-agent patterns.
- **agent-framework**: installs everything at once if you don't want to pick.

2.5 How one agent run works, step by step

Let's follow a single question through the system: "*What's the total for 2 units of P101?*"



Time flows from top to bottom. The framework does all the back-and-forth for you.

- **Step 1.** Your code calls *agent.run(...)* with the user's question.
- **Step 2.** The agent bundles the instructions, the conversation history and a description of every tool (name, docstring, parameters) and sends it to the model through the chat client.
- **Step 3.** The model reads the question, sees that *calculate_order_total* fits, and replies with a tool-call request containing the arguments.
- **Step 4.** MAF runs your actual Python function with those arguments.
- **Step 5.** Your function returns a string, such as "Total for 2 x Laptop: INR 110000."
- **Step 6.** MAF sends that result back to the model.
- **Step 7.** The model writes a friendly final answer, which *agent.run* returns to you.

> **Why your docstrings matter**
> In step 2 the model only sees each tool's name, docstring and parameter types, never your code. A clear docstring like "Calculate an order total when enough stock is available" is how the model knows when to use that tool. A vague docstring leads to wrong tool choices.

2.6 A complete example

Here is how your two shopping tools become a working agent. The tools live in *tools.py* (the file with *@tool*), and this is a separate *main.py*:

```
import asyncio
from dotenv import load_dotenv
from agent_framework import Agent
from agent_framework.openai import OpenAIChatClient
from tools import search_products, calculate_order_total

load_dotenv()   # reads OPENAI_API_KEY from your .env file


async def main():
    agent = Agent(
        client=OpenAIChatClient(),
        name="ShopAssistant",
        instructions=(
            "You help customers find products and price orders. "
            "Always use the tools; never guess prices or stock."
        ),
        tools=[search_products, calculate_order_total],
    )

    result = await agent.run("What's the total for 2 units of P101?")
    print(result.text)


asyncio.run(main())
```

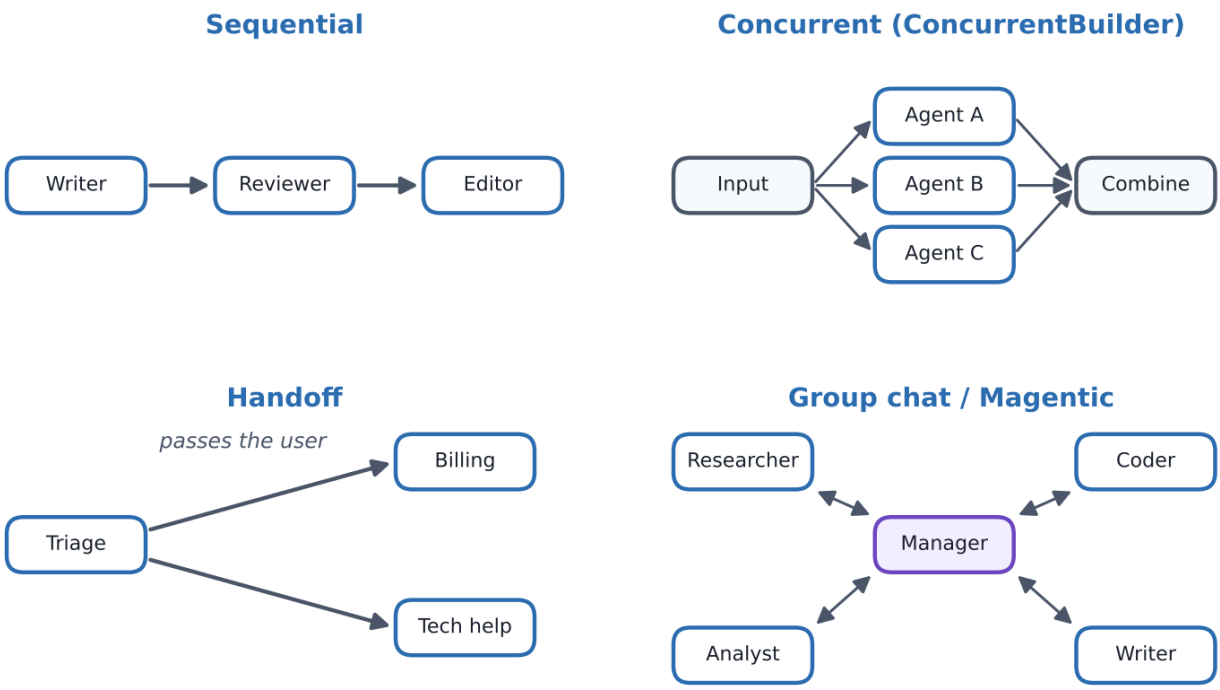
What each part does:

- **load_dotenv()** loads your API key so it isn't written in the code.
- **OpenAIChatClient()** is the model client from the architecture diagram.
- **instructions** is the agent's job description.
- **tools=[...]** hands the agent your two functions. This is the moment they stop being plain functions you test manually and become real agent tools.
- **agent.run(...)** runs the whole loop from section 2.5. It is *async*, which is why we use *await* and *asyncio.run*.

> **Version note**
> MAF is updated often. Parameter names can shift slightly between versions, so if something errors, compare with the official samples for the version in your requirements file.

2.7 Multi-agent orchestration

One agent with a few tools is enough for many tasks. For bigger jobs, it's often better to split the work across several specialised agents, just like a team of people. MAF gives you ready-made patterns for this:



A manager agent decides who speaks next and when the task is finished.

The main orchestration patterns in MAF.

Pattern	How it works	Good for
Sequential	Agents run one after another. Each one's output is the next one's input.	Write, then review, then edit.
Concurrent	The same input goes to several agents at the same time, and their answers are collected. Built with *ConcurrentBuilder*.	Getting several expert opinions quickly.
Handoff	One agent passes the whole conversation to the right specialist.	Customer support triage.
Group chat	Agents take turns in a shared conversation, guided by a manager.	Brainstorming, debate.
Magentic	A manager agent plans the task, assigns steps to workers and tracks progress until done.	Open-ended, multi-step research.

Under the hood, all of these are built on MAF's **workflow engine**, which connects steps as a graph. You can also build your own custom graph when none of the ready-made patterns fit.

2.8 Other features worth knowing

- **MCP support.** MCP (Model Context Protocol) is a standard way to share tools. Your agent can connect to an MCP server and use its tools without you writing them.
- **A2A.** Agent-to-agent protocol, so your agent can talk to agents built with other frameworks.
- **Observability.** Built-in OpenTelemetry tracing records every model call and tool call, which makes debugging much easier.
- **DevUI.** A local browser interface for chatting with and debugging your agents while you build.
- **Human-in-the-loop.** You can require a person to approve certain tool calls, such as placing a real order.

2.9 What MAF does NOT do for you

MAF runs wherever *you* run it. If you close VS Code, your agent stops. For a real product you still need somewhere to host it, scale it for many users, secure it, and monitor it. That's the gap Foundry Agent Service fills.

3. Part B: Microsoft Foundry Agent Service

3.1 First, what is Microsoft Foundry?

Microsoft Foundry (previously called **Azure AI Foundry**) is Microsoft's cloud platform for building AI applications on Azure. It brings together a large catalog of models, tools, knowledge sources, evaluation and deployment in one place, with a web portal and SDKs. If you see older tutorials say "Azure AI Foundry", it's the same platform under its old name.

3.2 What is Foundry Agent Service?

Foundry Agent Service is the part of Foundry dedicated to agents. Microsoft describes it as a fully managed platform for building, deploying and scaling AI agents. "Fully managed" means Azure handles the servers, scaling, storage of conversations, security and monitoring. You focus on what the agent should do.

In simple words: **Foundry Agent Service is a production home for your agents in Azure.**

3.3 Ways to build an agent in Foundry

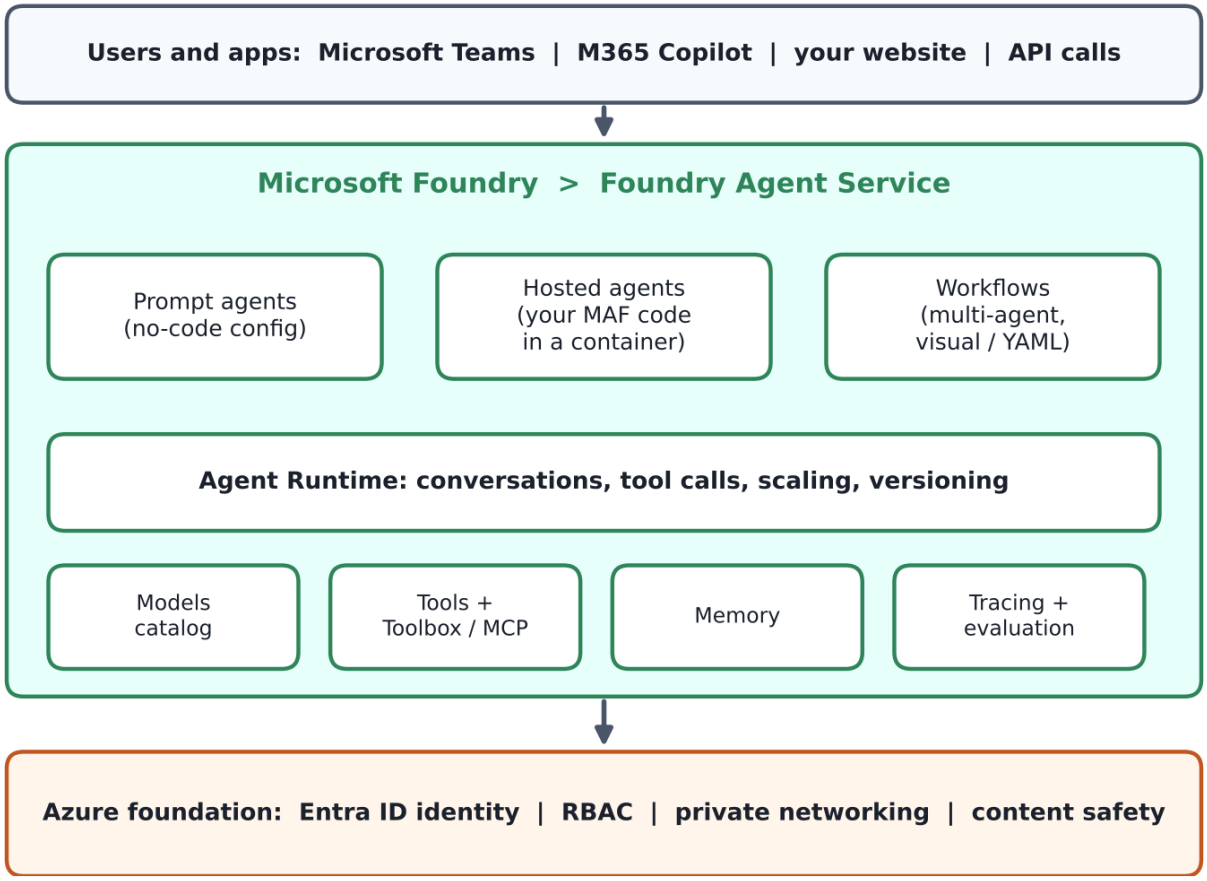
Foundry offers a spectrum from no-code to full code. You pick the one that matches how much control you need.

Type	How you build it	Who runs the logic	Best for
Prompt agents	Fill in instructions, pick a model and attach tools, in the Foundry portal or via SDK / REST API. No app code.	Foundry runs everything.	Getting started, internal tools, simple production agents.
Hosted agents	Write the agent in code with MAF (or LangGraph, OpenAI Agents SDK and others) and ship it as a container or a zip of source code.	Your code, running on Foundry's managed runtime.	Custom logic, multi-agent systems, full control.
Workflows	Connect agents and business steps in a visual builder in the portal, or in YAML from VS Code.	Foundry runs the workflow.	Multi-step processes without writing orchestration code.

For a MAF developer, **hosted agents** are the most important: they let you take the exact MAF code you wrote and tested locally and run it on Azure with a managed endpoint, automatic scaling and its own identity.

> **Feature status changes often**
> During 2026 several Foundry features (including hosted agents, workflows and memory) moved from preview toward general availability at different times. Always check the official Foundry Agent Service overview page for what is preview and what is GA today.

3.4 Foundry Agent Service architecture



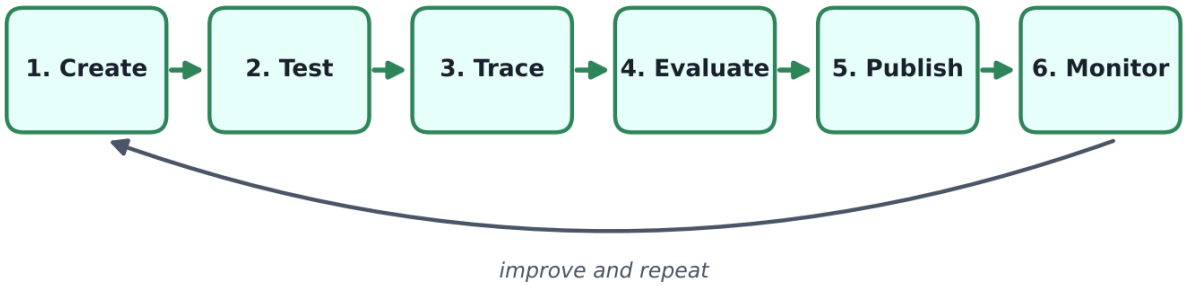
Foundry Agent Service architecture: agents on top of a managed runtime, shared services, and Azure's security foundation.

Reading the diagram from top to bottom:

- **Users and apps** reach your agent through a stable endpoint, or through places where people already work, like Microsoft Teams and Microsoft 365 Copilot.
- **The three agent types** (prompt, hosted, workflow) are the agents you create.
- **Agent Runtime** is the engine. It hosts and scales the agents and manages conversations, tool calls and versions. This is the part you would otherwise have to build and operate yourself.
- **Shared services:** the model catalog (many models, not only OpenAI), built-in tools such as web search, code interpreter, file search and MCP servers, managed memory so agents can remember across sessions, and tracing and evaluation to inspect and grade agent behaviour.
- **Azure foundation:** Microsoft Entra ID gives each agent its own identity, RBAC controls who can create or call agents, private networking keeps traffic inside your virtual network, and content safety filters harmful input and output.

3.5 The agent lifecycle in Foundry

Foundry supports the full journey from idea to production:

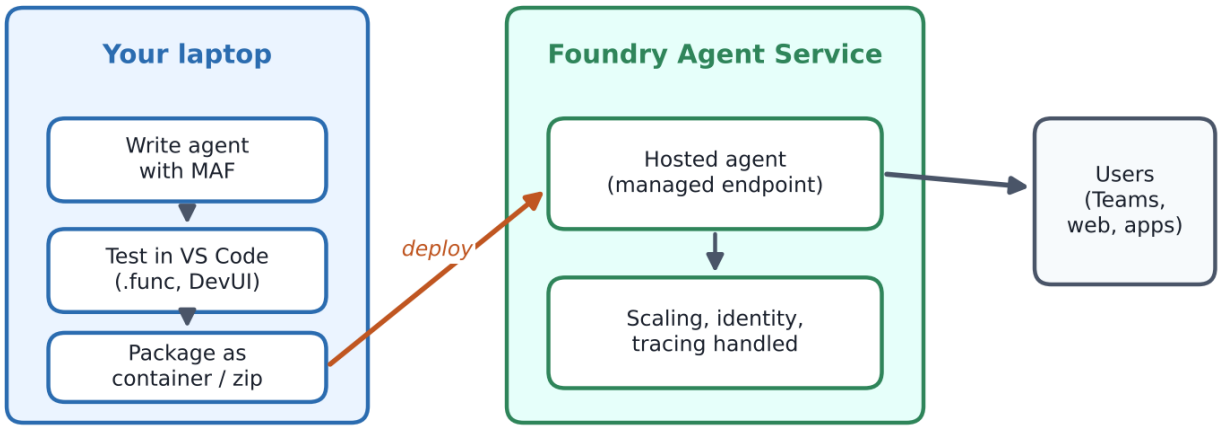


The build-test-deploy-monitor cycle.

- **1. Create:** define a prompt agent in the portal, or deploy a hosted agent from code.
- **2. Test:** chat with it in the agents playground, or run it locally.
- **3. Trace:** inspect every model call, tool call and decision it made.
- **4. Evaluate:** measure quality and safety against test cases before release.
- **5. Publish:** create a stable version and endpoint, and share it to Teams, M365 Copilot or the Entra Agent Registry.
- **6. Monitor:** watch real usage, errors and cost in dashboards, then improve and repeat.

4. How MAF and Foundry work together

MAF and Foundry Agent Service are not competitors. They are two halves of the same story, and you can use either one without the other.



Same MAF code: run it locally today, host it on Foundry later.

A typical path: build and test with MAF locally, then deploy to Foundry as a hosted agent.

Three common combinations

- **MAF only.** You build an agent and run it yourself, for example inside your FastAPI backend. Full control, but hosting, scaling and security are your job. This is what you are doing when you run code from VS Code.
- **Foundry only.** You create a prompt agent in the Foundry portal with no code at all. Fast, but limited to what the portal can configure.
- **MAF + Foundry.** You write the agent in MAF and deploy it to Foundry as a hosted agent, or your MAF code connects to an agent that lives in Foundry. You get code-level control plus Azure's managed runtime.

For the last option, MAF has a Foundry integration package (*agent-framework-foundry*). Its *FoundryAgent* class lets your Python code call a prompt agent defined in Foundry, using your project endpoint, the agent's name and version, and an Azure credential. The model, instructions and tools are stored on the Foundry side.

> **Where your current project fits**
> Building agents with *agent_framework*, *OpenAIChatClient* and *ConcurrentBuilder*, and running them from VS Code, is **MAF only**. You'd be using Foundry Agent Service only after deploying the agent to Foundry or creating it there.

5. Side-by-side comparison

Question	Microsoft Agent Framework	Foundry Agent Service
What is it?	An open-source SDK (code library)	A managed cloud platform on Azure
Main job	**Build** agents and multi-agent workflows	**Run**, scale, secure and monitor agents
How you use it	pip install, write Python or C#	Foundry portal, SDK or REST API
Where it runs	Anywhere: laptop, your server, any cloud	Azure
Who manages servers	You	Microsoft
Cost	Free library; you pay for model API calls and your own hosting	Pay for Azure usage (models, runtime, tools)
Multi-agent	Built-in patterns: sequential, concurrent, handoff, group chat, magentic	Workflows (visual or YAML), or host your MAF orchestration
Security	You implement it	Built-in Entra identity, RBAC, private networking, content safety
Monitoring	OpenTelemetry traces you send to a tool of your choice	Built-in tracing, evaluation and dashboards
Best stage	Learning, prototyping, custom logic	Production and enterprise deployment

When to use which

- **Learning or building a project for your portfolio?** Start with MAF locally. It teaches you how agents really work.
- **Need a quick internal agent with standard tools and no custom logic?** A Foundry prompt agent may be enough.
- **Need custom logic AND enterprise hosting?** Build with MAF, deploy as a Foundry hosted agent.
- **Not on Azure?** MAF still works anywhere; Foundry Agent Service is Azure-specific.

6. Interview questions and answers

Q. What is Microsoft Agent Framework?

It's Microsoft's open-source SDK for building AI agents and multi-agent workflows in Python and .NET. It combines the enterprise foundations of Semantic Kernel with the multi-agent ideas from AutoGen, and reached version 1.0 in April 2026.

Q. What is Foundry Agent Service?

It's a fully managed Azure platform for deploying, scaling and governing AI agents. It provides the runtime, identity, security, tracing and publishing, so teams don't have to build that infrastructure themselves.

Q. What's the difference between them?

MAF is for building agents in code; Foundry Agent Service is for running and managing them in production. You can build with MAF and deploy to Foundry as a hosted agent.

Q. How does an agent decide which tool to call?

The framework sends the model each tool's name, description and parameter schema. The model picks a tool and arguments, the framework runs the function, and the result goes back to the model. That's why clear docstrings matter.

Q. What does ConcurrentBuilder do?

It builds a concurrent orchestration where the same input goes to several agents in parallel, and their results are collected. It's faster than running them one after another when the agents don't depend on each other.

Q. How did you test your tools?

I tested each tool as a normal function first, calling it directly with valid and invalid inputs, before connecting it to an agent. That separates bugs in my code from mistakes the model makes.

> **A tip on honesty**
> Only claim what you actually built, and be ready to walk through your own code. Interviewers usually ask follow-up questions, and explaining your real project clearly is worth more than a polished sentence.

7. Quick glossary

Term	Meaning
Agent	An AI program that uses a model, instructions and tools to complete tasks, not just chat.
LLM	Large language model, the "brain" of the agent, such as a GPT model.
Tool	A function the model can ask the framework to run, like *search_products*.
@tool	MAF decorator that turns a Python function into an agent tool.
Model client	The MAF component that connects to an LLM provider, like *OpenAIChatClient*.
Orchestration	Coordinating several agents to work on one task.
MCP	Model Context Protocol, a standard way to share tools between AI apps.
A2A	Agent-to-agent protocol for agents from different frameworks to talk.
Prompt agent	A Foundry agent defined only by configuration (instructions, model, tools).
Hosted agent	Your own agent code (for example MAF) running as a container on Foundry.
Entra ID	Microsoft's identity service; gives agents and users secure identities in Azure.
RBAC	Role-based access control: who is allowed to do what.
Tracing	A recorded timeline of every step an agent took, used for debugging.

Further reading (official sources)

- Microsoft Agent Framework documentation: learn.microsoft.com/agent-framework
- Source code and samples: github.com/microsoft/agent-framework
- Foundry Agent Service overview: learn.microsoft.com/azure/foundry/agents/overview
- Python package: pypi.org/project/agent-framework

Both products change quickly. Facts in this guide reflect public information as of November 2026; check the official docs for the latest feature status.